

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Docente:
Prof. José Canelón, Ph.D.

UNIDAD II

RED PERCEPTRÓN MULTICAPA

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

Contenido

- Redes neuronales shallow
- Funciones de activación
- Red perceptrón multicapa (MLP)
- Retropropagación y regla de la cadena
- Regularización
- Optimizadores. Learning rate scheduling. Inicialización de pesos
- Debugging: gradient flow, vanishing/exploding gradients, sanity checks.
- Monitoreo de experimentos con Weights & Biases

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

Redes Neuronales

- Una red neuronal es un sistema de procesamiento paralelo, masivo y distribuido, formado por unidades simples que tienen la capacidad de adquirir conocimiento a través de un proceso de entrenamiento, y almacenar este conocimiento para su uso posterior.
- Existen diferentes tipos de redes neuronales, entre los que pueden mencionarse
 - Redes perceptrón multicapa
 - Redes convolucionales
 - Redes recurrentes
 - Transformers

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

Redes Perceptrón Multicapa

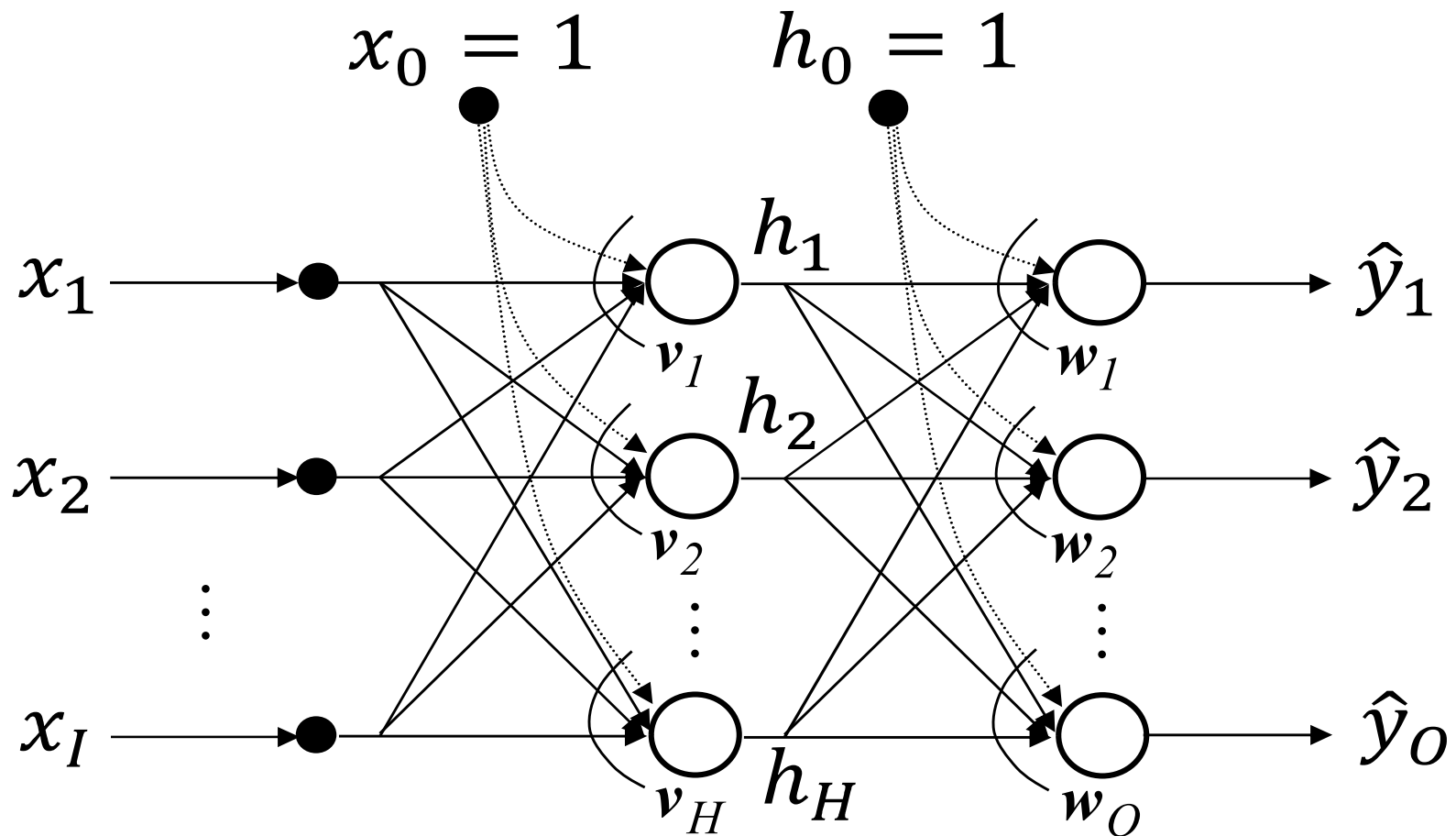
- También se conocen como Redes Feedforward o Redes Densas
- Están constituidas por varias capas de perceptrones (nodos), conectadas en forma secuencial.
- Dos capas consecutivas están completamente conectadas
- Existen tres tipos de capas:
 1. capa de entrada, en la que se aplica el vector de entrada de la red.
 2. capa de salida, en la que aparece el vector de salida que produce la red.
 3. capas ocultas, que se ubican entre la capa de entrada y la capa de salida. Puede haber una o más capas ocultas

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

Redes Perceptrón Multicapa

Una red shallow tiene una capa oculta entre la entrada y la salida



INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

Redes Perceptrón Multicapa

Pesos entre la capa de entrada y la capa oculta

$$\mathbf{V} = \begin{bmatrix} v_{10} & v_{11} & v_{12} & \cdots & v_{1I} \\ v_{20} & v_{21} & v_{22} & \cdots & v_{2I} \\ \vdots & \vdots & \vdots & & \vdots \\ v_{H0} & v_{H1} & v_{H2} & \cdots & v_{HI} \end{bmatrix} \in \mathbb{R}^{H \times (I+1)}$$

Pesos entre la capa oculta y la capa de salida

$$\mathbf{W} = \begin{bmatrix} w_{10} & w_{11} & w_{12} & \cdots & w_{1H} \\ w_{20} & w_{21} & w_{22} & \cdots & w_{2H} \\ \vdots & \vdots & \vdots & & \vdots \\ w_{O0} & w_{O1} & w_{O2} & \cdots & w_{OH} \end{bmatrix} \in \mathbb{R}^{O \times (H+1)}$$

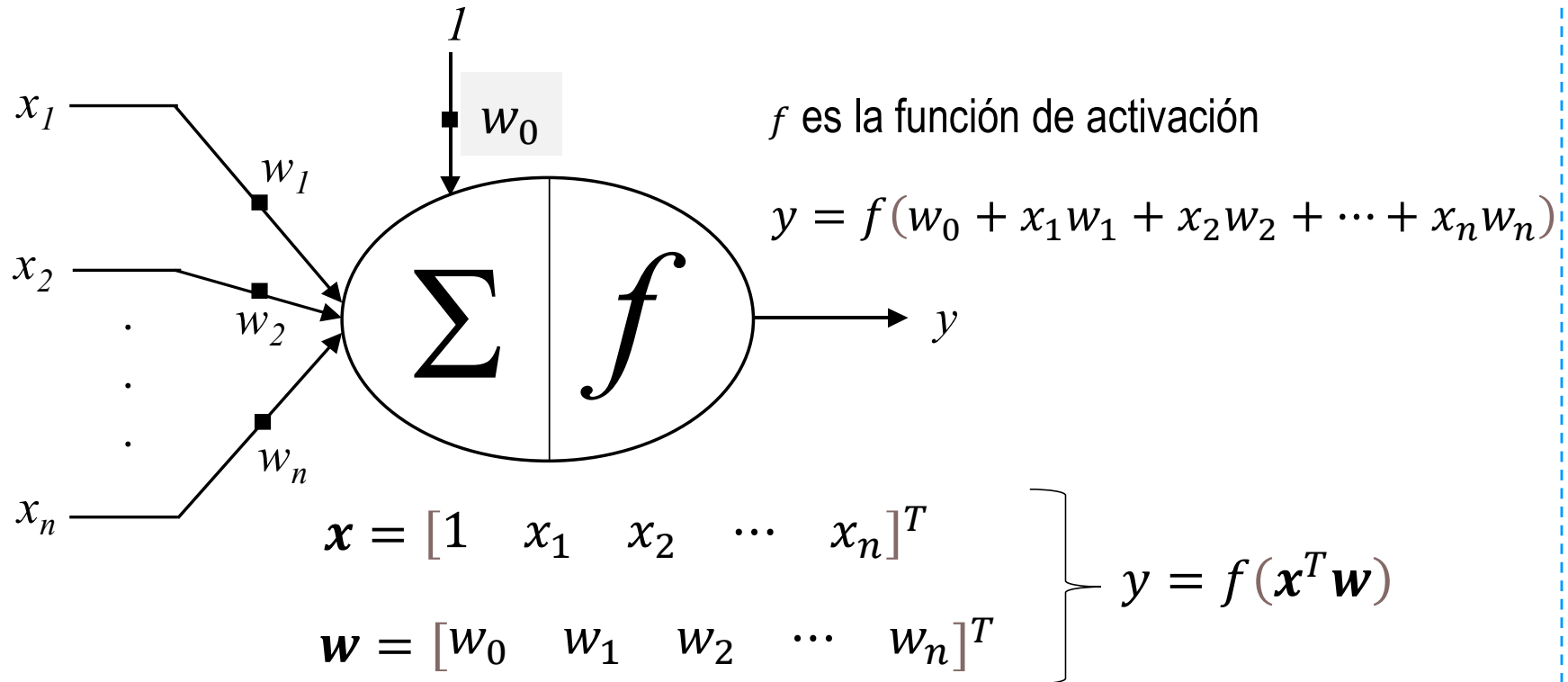
Número total de pesos: $H \times (I + 1) + O \times (H + 1)$

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

Redes Perceptrón Multicapa El Perceptrón

- Unidad computacional básica inspirada en la neurona biológica (McCulloch & Pitts, 1943)



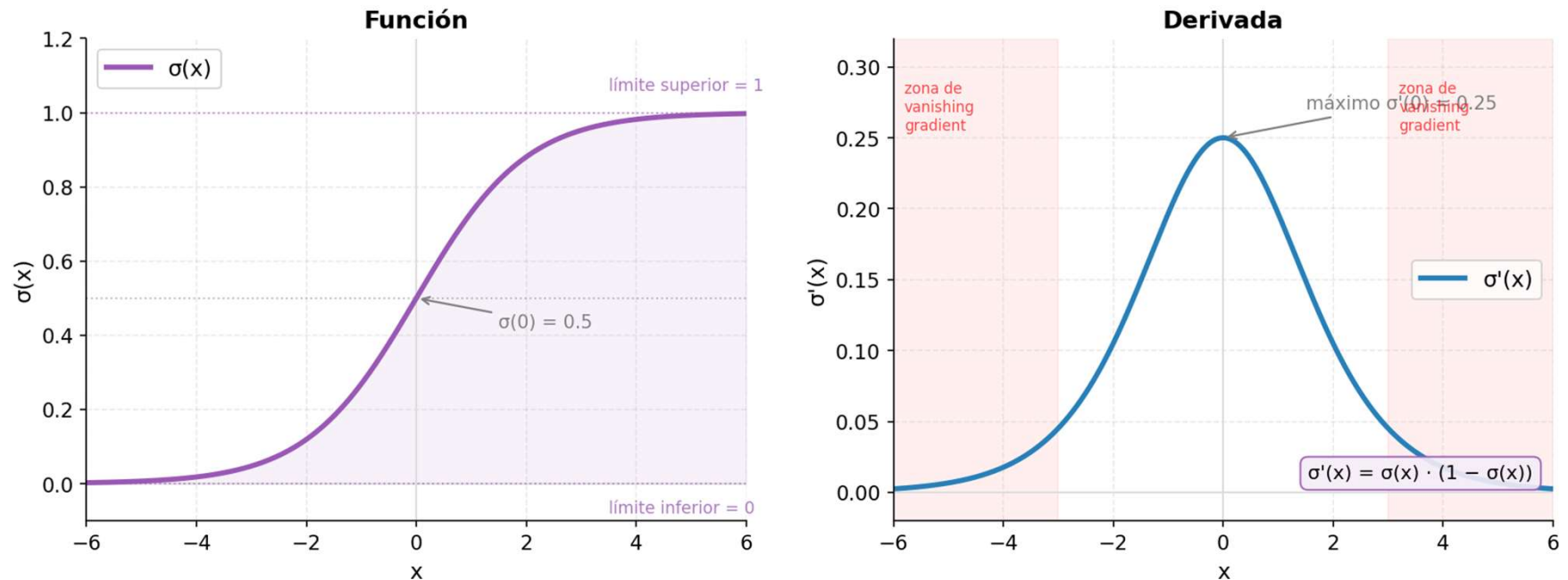
Redes Perceptrón Multicapa El Perceptrón

- La función de activación $f(\cdot)$ debe ser no lineal
 - Si $f(\cdot)$ no es no lineal: la red completa colapsa a una transformación lineal
 - Si $f(\cdot)$ es no lineal: la red puede representar funciones arbitrariamente complejas

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

Redes Perceptrón Multicapa Funciones de Activación Sigmoide

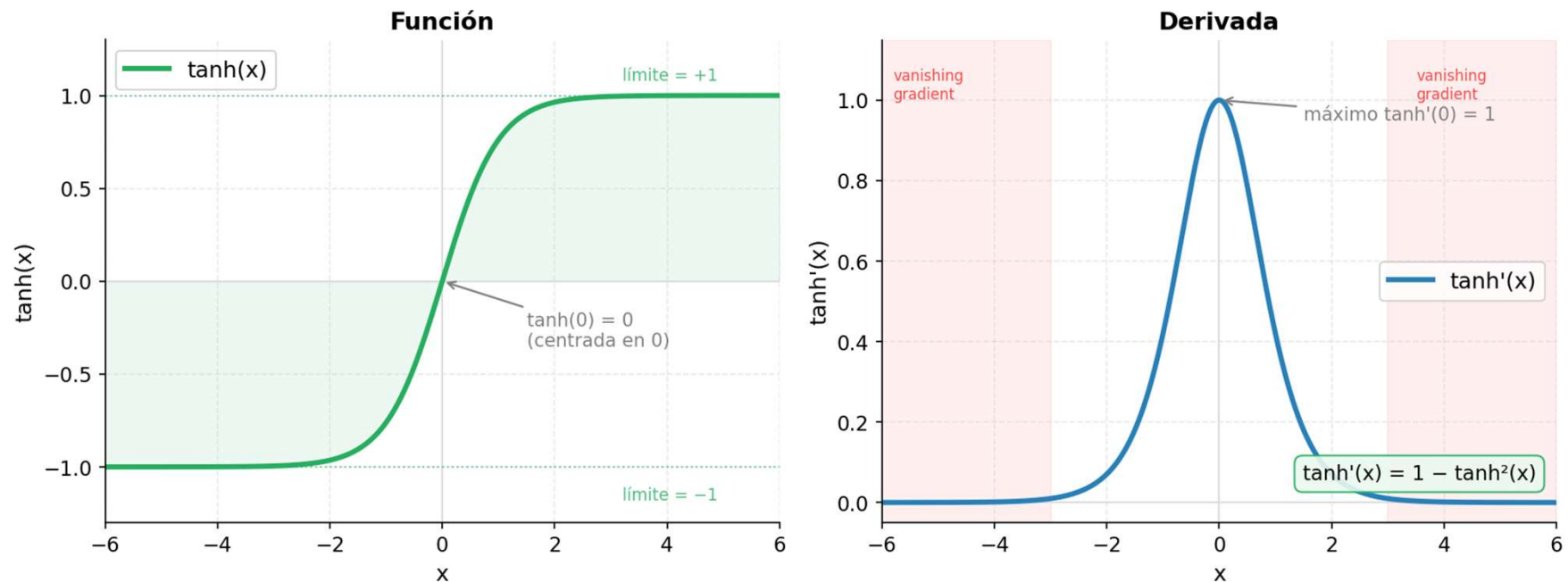


$$f(x) = \frac{1}{1 + e^{-x}} \Rightarrow f'(x) = \frac{e^{-x}}{(1 + e^{-x})^2}$$

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

Redes Perceptrón Multicapa Funciones de Activación Tangente hiperbólica

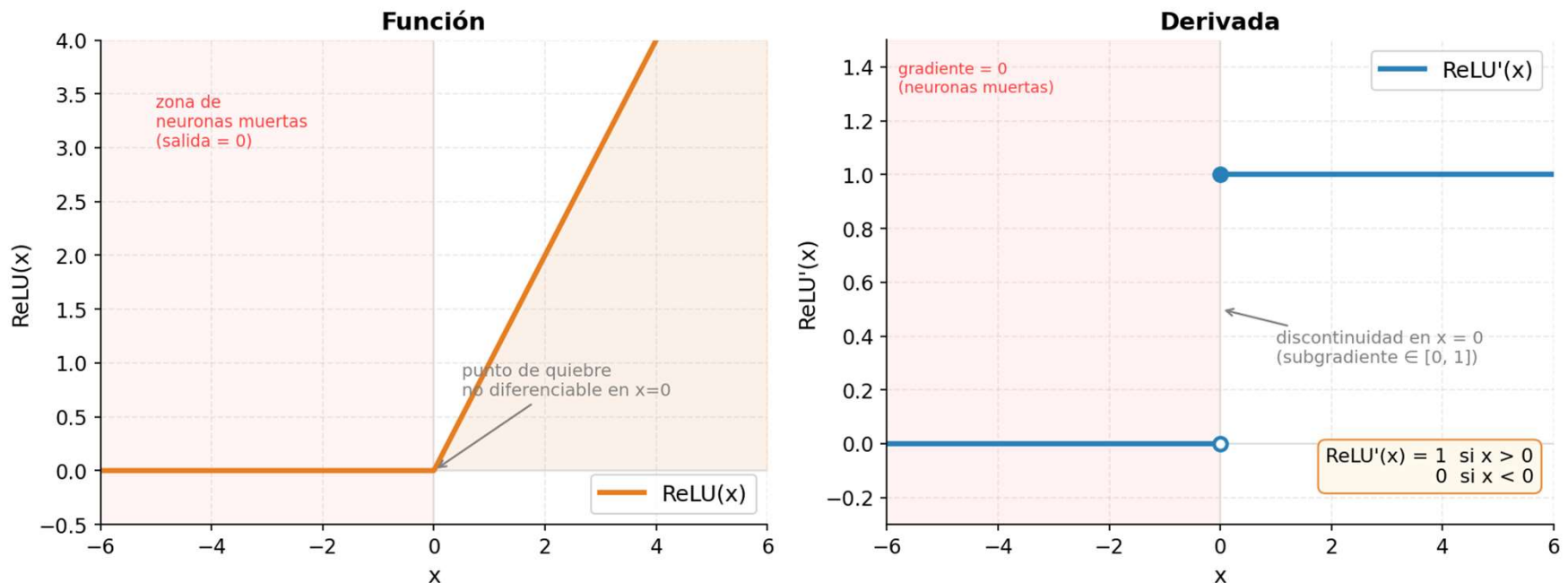


$$f(x) = \tanh(x) \Rightarrow f'(x) = \text{sech}^2 x$$

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

Redes Perceptrón Multicapa Funciones de Activación Rectified Linear Unit (ReLU)

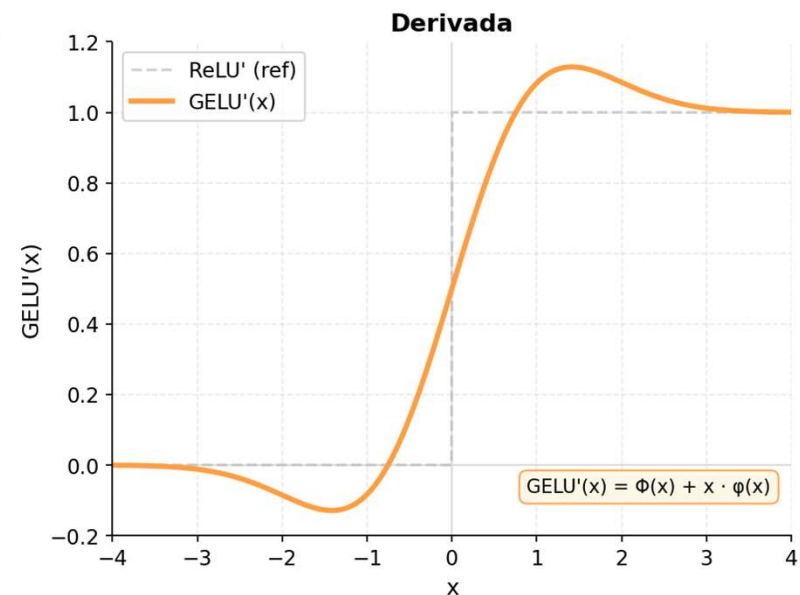
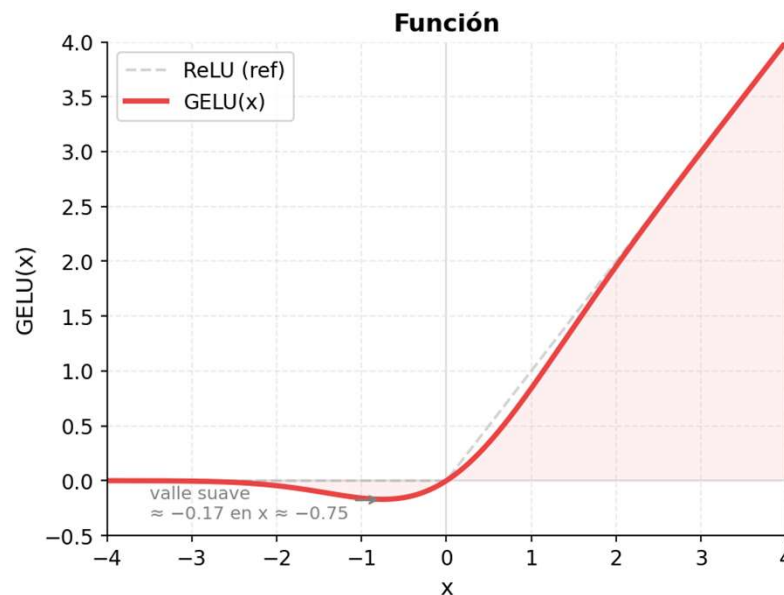


$$f(x) = \begin{cases} 0 & \text{para } x < 0 \\ x & \text{para } x > 0 \end{cases} \Rightarrow f'(x) = \begin{cases} 0 & \text{para } x < 0 \\ 1 & \text{para } x > 0 \end{cases}$$

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

Redes Perceptrón Multicapa Funciones de Activación Gaussian Error Linear Unit (GELU)



Ecuación
Exacta

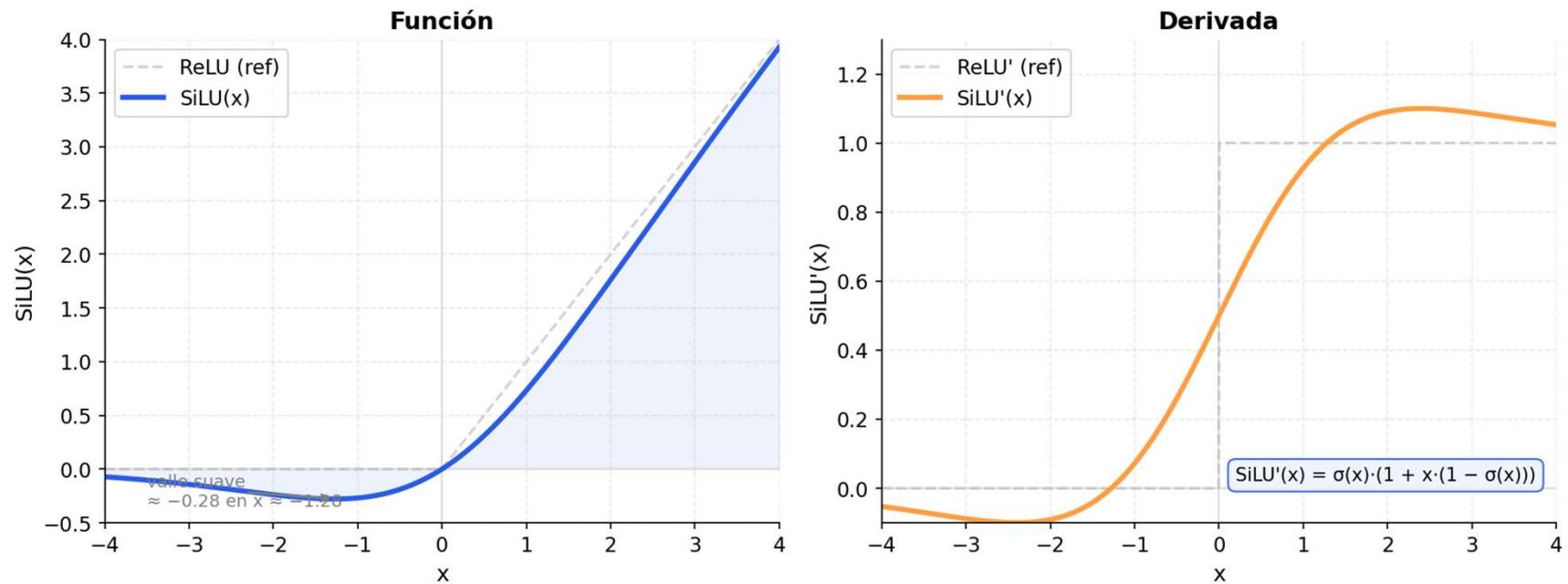
$$GELU(x) = x \cdot \Phi(x)$$

Ecuación
Aproximada
(PyTorch)

$$GELU(x) \approx \frac{x}{2} \left[1 + \tanh \left(\sqrt{\frac{2}{\pi}} (x + 0.044715 x^3) \right) \right]$$

$$GELU'(x) = \Phi(x) + x \phi(x)$$

Redes Perceptrón Multicapa Funciones de Activación Sigmoid Linear Unit (SiLU/Swish)



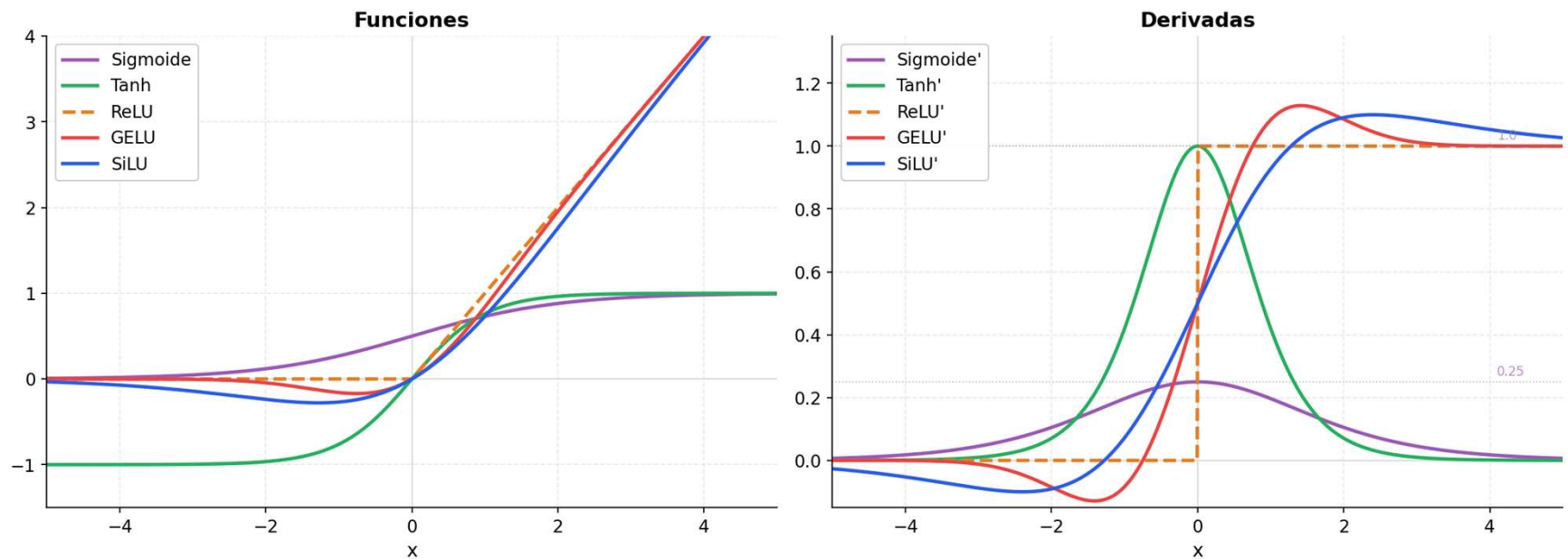
$$SiLU(x) = \frac{x}{1 + e^{-x}} \Rightarrow SiLU'(x) = SiLU(x) + \sigma(x)(1 - SiLU(x))$$

donde $\sigma(x)$ es la sigmoide

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

Redes Perceptrón Multicapa Comparativa de Funciones de Activación



INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

Redes Perceptrón Multicapa Comparativa de Funciones de Activación

	ReLU	GELU	SiLU
Diferenciable en 0	No	Sí	Sí
Valores negativos	Exactamente 0	≈ 0 (suave)	≈ 0 (suave)
Costo computacional	Muy bajo	Medio	Bajo
Uso principal	CNNs clásicas RPMC	Transformers (NLP/ViT)	CNNs modernas, YOLO

La ventaja clave de SiLU y GeLU sobre ReLU es que son **suaves y no monótonas**: tienen un pequeño "valle" para valores levemente negativos (~ -0.1 para $x \approx 0.5$). Esto empíricamente mejora la capacidad expresiva de la red y el flujo de gradientes durante el entrenamiento.

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

Redes Perceptrón Multicapa Elección de la Función de Activación

- La elección de la función de activación ha sido uno de los factores más determinantes en la evolución del Deep Learning

Cronología

- 1943 — McCulloch & Pitts: neurona binaria (función escalón)
- 1958 — Rosenblatt: Perceptrón con función escalón
- 1986 — Rumelhart et al.: backpropagation con Sigmoides (diferenciable)
- 1990s-2000s — *tanh* reemplaza a Sigmoides en capas ocultas (Lecun et al., 1998)
- 2010 — Nair & Hinton: *ReLU* muestra clara ventaja sobre *tanh* en CNNs
- 2012 — AlexNet: *ReLU* + Dropout → revolución en ImageNet
- 2016 — Clevert et al.: ELU; He et al.: Leaky *ReLU* con análisis
- 2016 — Hendrycks & Gimpel: GELU — adoptada por BERT, GPT y todos los Transformers
- 2017 — Ramachandran et al.: Swish (SiLU) via búsqueda automática

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

Problemas de la Sigmoide en Redes Profundas

Problema 1: Vanishing gradient

- Para $|x| > 3$, el gradiente prácticamente desaparece
- En backpropagation: el gradiente se multiplica en cada capa
- Con L capas: gradiente $\approx (0.25)^L \rightarrow$ exponencialmente pequeño
- Con 10 capas: gradiente $\approx (0.25)^{10} \approx 10^{-6}$ — las capas iniciales no aprenden

Problema 2: Salida no centrada en 0

- $\sigma(x) \in (0, 1)$ siempre positivo
- Gradientes de los pesos de la capa siguiente son siempre del mismo signo
- Provoca actualizaciones 'en zigzag' en el espacio de parámetros

Problema 3: Costo computacional

- e^x es más costoso que $\max(0, x)$ de *ReLU*

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

Problemas de la Sigmoide en Redes Profundas

¿Cuándo aún se usa?

- Capa de salida para clasificación binaria
- Gates en LSTMs (con $\tanh$ para cell state)
- Attention weights (junto con softmax)

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

Función de Activación tanh

Ventaja sobre Sigmoides

- Output centrado en 0 $\rightarrow$ gradientes sin sesgo de signo
- Derivada mayor (hasta 1.0 vs 0.25) $\rightarrow$ entrenamiento más rápido

Mismo problema: vanishing gradient

- Para $|x| > 2$, $\tanh'(x) \approx 0$ — sigue saturando
- Menos grave que sigmoides pero presente en redes muy profundas

Uso actual

- Cell state y output gate en LSTMs y GRUs (Unidad VI del curso)
- Redes shallow para tareas con salida en $(-1, 1)$

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

Vanishing Gradients

Consecuencia práctica

- Las primeras capas reciben gradientes casi nulos
- Sus pesos apenas se actualizan — no aprenden representaciones útiles
- El entrenamiento efectivo se limita a las últimas capas

Soluciones modernas

- ReLU y variantes — no satura para $x > 0$
- Residual connections (ResNet) — bypass que preserva el gradiente
- Batch Normalization — mantiene activaciones en zona activa

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

¿Por qué ReLU funciona mejor que Sigmoid/Tanh?

- No satura para $x > 0$: la derivada es exactamente 1, no se encoge
- Sparse activation: con datasets típicos, ~50% de neuronas están inactivas
 - Con sparsidad, cada sample activa un subconjunto distinto → los conceptos están más separados en el espacio de activaciones
- Eficiencia computacional: $\max(0, x)$ es trivial en GPU

Referencia histórica

- Popularizada por Nair & Hinton (2010); consolidada con AlexNet (Krizhevsky, 2012)
- 'Using ReLU instead of sigmoid/tanh makes training much faster' — Krizhevsky et al

ReLU — El Problema de las Neuronas Muertas

¿Qué es una neurona muerta?

- Una neurona ReLU queda 'muerta' cuando su preactivación es siempre negativa
- Condición: $w^T x + b < 0$ para todas las muestras de entrenamiento
- Consecuencia: salida = 0, gradiente = 0 $\rightarrow$ los pesos no se actualizan nunca más

Causas comunes

- Learning rate muy alto: actualización excesiva que lleva b a valores muy negativos
- Inicialización incorrecta de pesos
- Datos con alta varianza sin normalización previa

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

ReLU — El Problema de las Neuronas Muertas

Impacto práctico

- En redes bien entrenadas: 10-50% de neuronas pueden estar inactivas (normal y esperado)
- Si >80% están muertas: capacity collapse — la red no puede representar bien los datos

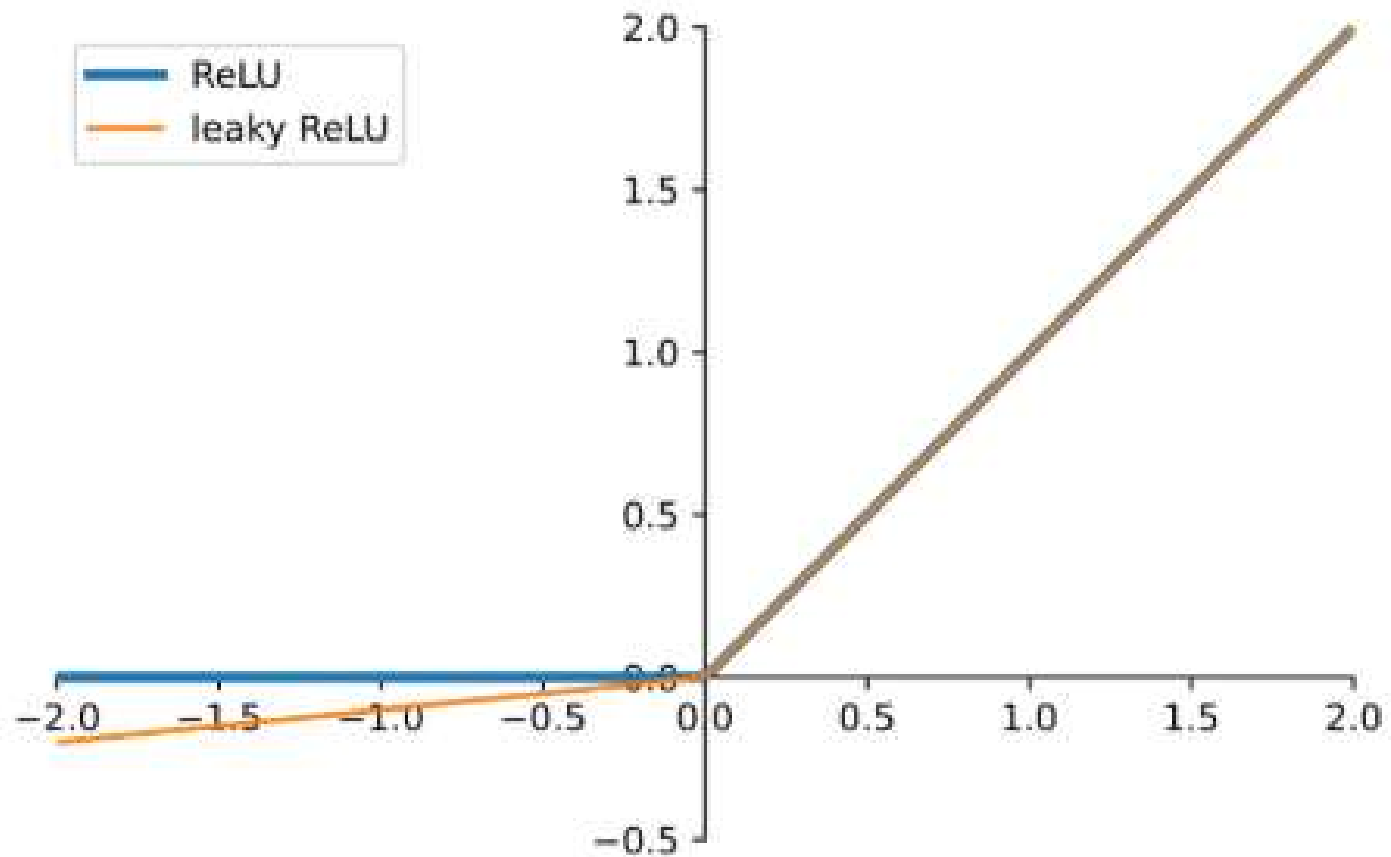
Soluciones

- Leaky ReLU: $f(x) = \max(\alpha x, x)$ con $\alpha = 0.01$ — pequeño gradiente para $x < 0$
- ELU: $f(x) = x$ si $x \geq 0$, $\alpha(e^x - 1)$ si $x < 0$ — suave y continua
- Inicialización cuidadosa (He initialization)
- Batch Normalization: mantiene activaciones en rango activo

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

Leaky ReLU

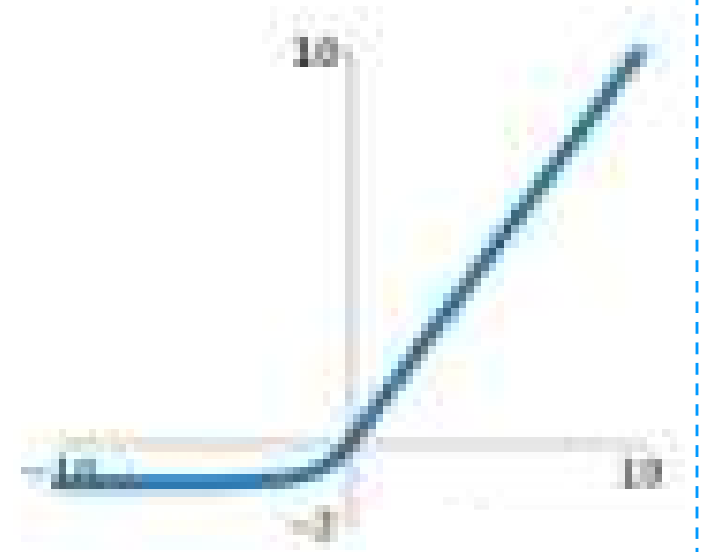


INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

¿Por qué GELU y SiLU Superan a ReLU?

1. Suavidad y diferenciabilidad

- ReLU no es diferenciable en $x=0$ — puede crear inestabilidades en optimización
- GELU y SiLU son C^∞ — gradientes suaves en toda la recta real

2. No monotonía — mayor capacidad expresiva

- El pequeño valle negativo añade curvatura — la red puede representar más funciones con menos neuronas
- Análogo a agregar una pequeña no linealidad extra sin costo de parámetros

3. Gradiente siempre no nulo

- No hay zona donde el gradiente sea exactamente 0 (excepto $x \rightarrow -\infty$)
- Elimina completamente el problema de neuronas muertas

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

¿Por qué GELU y SiLU Superan a ReLU?

4. Derivada puede superar 1

- Para $x \approx 1.5$: $\text{GELU}'(x) \approx \text{SiLU}'(x) \approx 1.1$
- Facilita el flujo de gradientes en redes muy profundas

Evidencia empírica

- En Transformers: GELU mejora consistentemente sobre ReLU
- En CNNs de visión: SiLU gana +0.3-0.5% accuracy sobre ReLU

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

Comparativa — Saturación

¿Qué es la saturación?

- Una función se satura cuando su derivada tiende a 0 para $|x| \rightarrow \infty$
- La saturación frena el aprendizaje: los gradientes desaparecen

Funciones que saturan

- Sigmoides: $\sigma'(x) \rightarrow 0$ para $|x| > 3$ (satura en ambos extremos)
- Tanh: $\tanh'(x) \rightarrow 0$ para $|x| > 2$ (satura en ambos extremos)

Funciones que no saturan (o saturan solo por un lado)

- ReLU: no satura para $x > 0$, pero tiene zona muerta para $x < 0$
- Leaky ReLU / ELU: no saturan en ningún extremo
- GELU / SiLU: no saturan superiormente ($\rightarrow x$ para $x \gg 0$)
- GELU / SiLU: sí saturan inferiormente ($\rightarrow 0$ para $x \rightarrow -\infty$)

Implicación práctica

- Sigmoides / Tanh: requieren inicialización y normalización cuidadosa para redes profundas
- ReLU y variantes: entrenamiento más estable en redes profundas (> 5 capas)

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

Comparativa — Flujo de Gradientes

Gradiente durante backpropagation

- El producto de matrices puede explotar o desvanecerse exponencialmente

Derivadas en $x=0$ (punto de operación típico con BN)

- Sigmoides: $\sigma'(0) = 0.25$ ← pequeño, contribuye a vanishing
- Tanh: $\tanh'(0) = 1.0$ ← neutro, mejor que sigmoide
- ReLU: $\text{ReLU}'(0) = 0$ o 1 ← riesgo de neuronas muertas
- GELU: $\text{GELU}'(0) = 0.5$ ← suave, gradiente moderado
- SiLU: $\text{SiLU}'(0) = 0.5$ ← igual que GELU en 0

Gradiente máximo

- Sigmoides: 0.25 — vanishing inevitable en redes profundas
- Tanh: 1.0 — neutro en 0, pero satura rápido
- ReLU: 1.0 para $x > 0$ — sin vanishing en zona activa
- GELU / SiLU: ~ 1.1 para $x \approx 1.5$ — leve amplificación, buena para flujo

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

Comparativa — Simetría y Centrado en 0

¿Por qué importa que la activación esté centrada en 0?

- Si $f(x) \geq 0$ siempre (Sigmoides, ReLU), los gradientes tienen todos el mismo signo
- Esto hace que las actualizaciones de los pesos de la capa siguiente sean todas del mismo signo
- Provoca movimiento en zigzag durante la optimización (Lecun et al., 1998)

Análisis por función

- Sigmoides: salida $\in (0,1)$ — no centrada, bias de gradiente
- Tanh: salida $\in (-1,1)$ — centrada en 0, mejor que sigmoides
- ReLU: salida $\in [0,\infty)$ — no centrada, media > 0
- Leaky ReLU/ELU: media ≈ 0 con inicialización correcta
- GELU / SiLU: salida puede ser ligeramente negativa — semicentrada

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

Comparativa — Simetría y Centrado en 0

Solución práctica

- Batch Normalization normaliza activaciones a media ≈ 0 después de cada capa
- Elimina la necesidad de preocuparse por el centrado de la activación
- Razón por la que ReLU funciona bien pese a no estar centrada

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

Comparativa — Costo Computacional

Costo relativo (operaciones por elemento)

- ReLU: $\max(0, x)$ → 1 comparación — costo mínimo
- Leaky ReLU: $\max(\alpha x, x)$ → 1 comparación + 1 multiplicación
- Sigmoid: $1/(1+\exp(-x))$ → 1 exp + operaciones básicas
- Tanh: $(\exp(x)-\exp(-x))/(\dots)$ → 2 exp + operaciones básicas
- ELU: $\exp(x)-1$ para $x < 0$ → 1 exp (solo para $x < 0$)
- SiLU: $x \cdot \sigma(x)$ → 1 exp + multiplicación
- GELU (exact): $x \cdot \text{erf}(x/\sqrt{2})$ → erf costoso
- GELU (approx): $x \cdot \tanh(\dots)$ → 2 exp + multiplicaciones

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

Comparativa — Costo Computacional

En la práctica

- La diferencia es despreciable comparada con las multiplicaciones matriciales dominantes
- ReLU sigue siendo preferida cuando el cómputo es muy limitado
- GELU/SiLU preferidas cuando la precisión importa más (NLP, visión de alta capacidad)

Aceleración hardware

- GPUs modernas tienen instrucciones especializadas para ReLU y sigmoide
- La diferencia real en benchmarks es $< 5\%$ del tiempo de forward pass

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

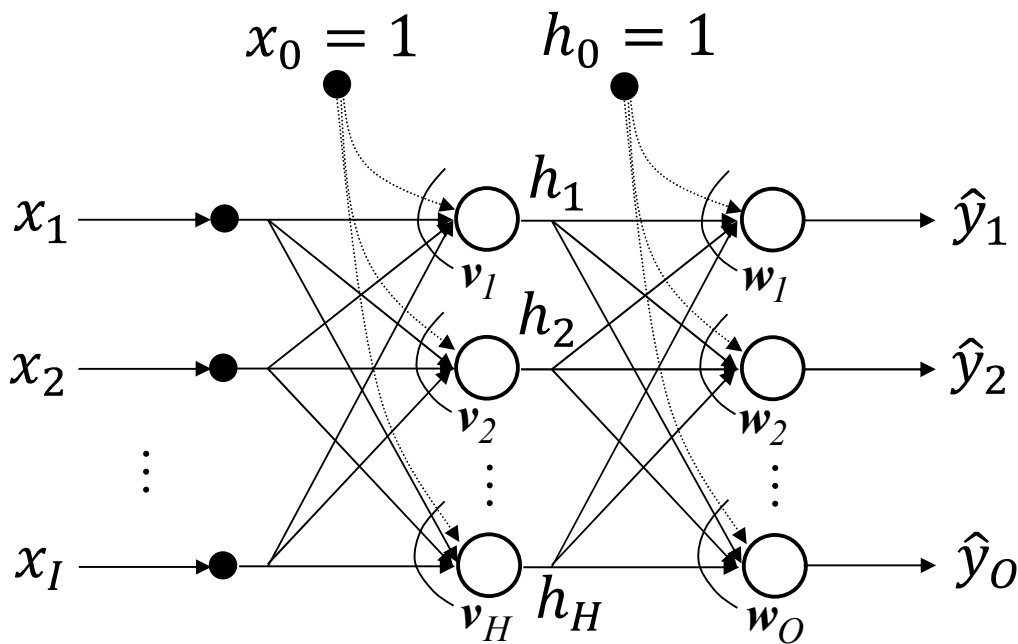
Tabla Comparativa — Funciones de Activación

Función	Rango	Satura	Centrada	Grad. máx.	Uso principal
Sigmoide	$(0,1)$	Ambos lados	No	0.25	Salida binaria, LSTM gates
Tanh	$(-1,1)$	Ambos lados	Sí	1.0	LSTM, GRU, redes shallow
ReLU	$[0,\infty)$	$x < 0$ muerto	No	1.0 ($x > 0$)	CNNs, MLPs clásicos
Leaky ReLU	$(-\infty,\infty)$	No	$\approx$ Sí	1.0	Alternativa a ReLU
ELU	$(-\alpha,\infty)$	$x \rightarrow -\infty$	$\approx$ Sí	1.0	Redes medianas, tabular
GELU	$\approx(-0.17,\infty)$	$x \rightarrow -\infty$	$\approx$ Sí	~ 1.1	Transformers: BERT, GPT
SiLU/Swish	$\approx(-0.28,\infty)$	$x \rightarrow -\infty$	$\approx$ Sí	~ 1.1	YOLO, EfficientNet

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

Redes Perceptrón Multicapa



$$\mathbf{x} = [x_1 \quad x_2 \quad x_3 \quad \cdots \quad x_I]^T$$

$$\mathbf{x}' = [1 \quad x_1 \quad x_2 \quad \cdots \quad x_I]^T$$

$$\mathbf{h} = f_h(\mathbf{V}\mathbf{x}')$$

$$\mathbf{h} = [h_1 \quad h_2 \quad \cdots \quad h_H]^T$$

$$\mathbf{h}' = [1 \quad h_1 \quad h_2 \quad \cdots \quad h_H]^T$$

$$\hat{\mathbf{y}} = f_y(\mathbf{W}\mathbf{h}')$$

$$\hat{\mathbf{y}} = [\hat{y}_1 \quad \hat{y}_2 \quad \hat{y}_3 \quad \cdots \quad \hat{y}_O]^T$$

A partir de estas ecuaciones, la función que implementa la red puede expresarse (en forma compacta) como $\hat{\mathbf{y}} = \mathcal{F}(\mathbf{x}, \Theta)$ donde Θ es un vector que contiene todos los pesos de la red

Redes Perceptrón Multicapa Teorema de Aproximación Universal

- **Enunciado (Cybenko, 1989 · Hornik, 1991)**

Una red neuronal shallow con suficientes neuronas ocultas y una función de activación no lineal continua puede aproximar cualquier función continua

$$f: \mathbb{R}^D \rightarrow \mathbb{R}$$

con precisión arbitraria sobre un dominio compacto

Condición sobre la función de activación

- Cybenko (1989): demostró para activaciones sigmoidales
- Hornik (1991): generalizó a cualquier activación no constante, acotada y continua
- Leshno et al. (1993): condición mínima = f no polinomial

Redes Perceptrón Multicapa **Teorema de Aproximación Universal**

- **Enunciado (Cybenko, 1989 · Hornik, 1991)**

Implicación práctica

- En principio, una sola capa oculta es suficiente para representar cualquier función
- El problema no es qué puede representar, sino cuántas neuronas necesita
- Para algunas funciones necesita exponencialmente más neuronas que una red profunda

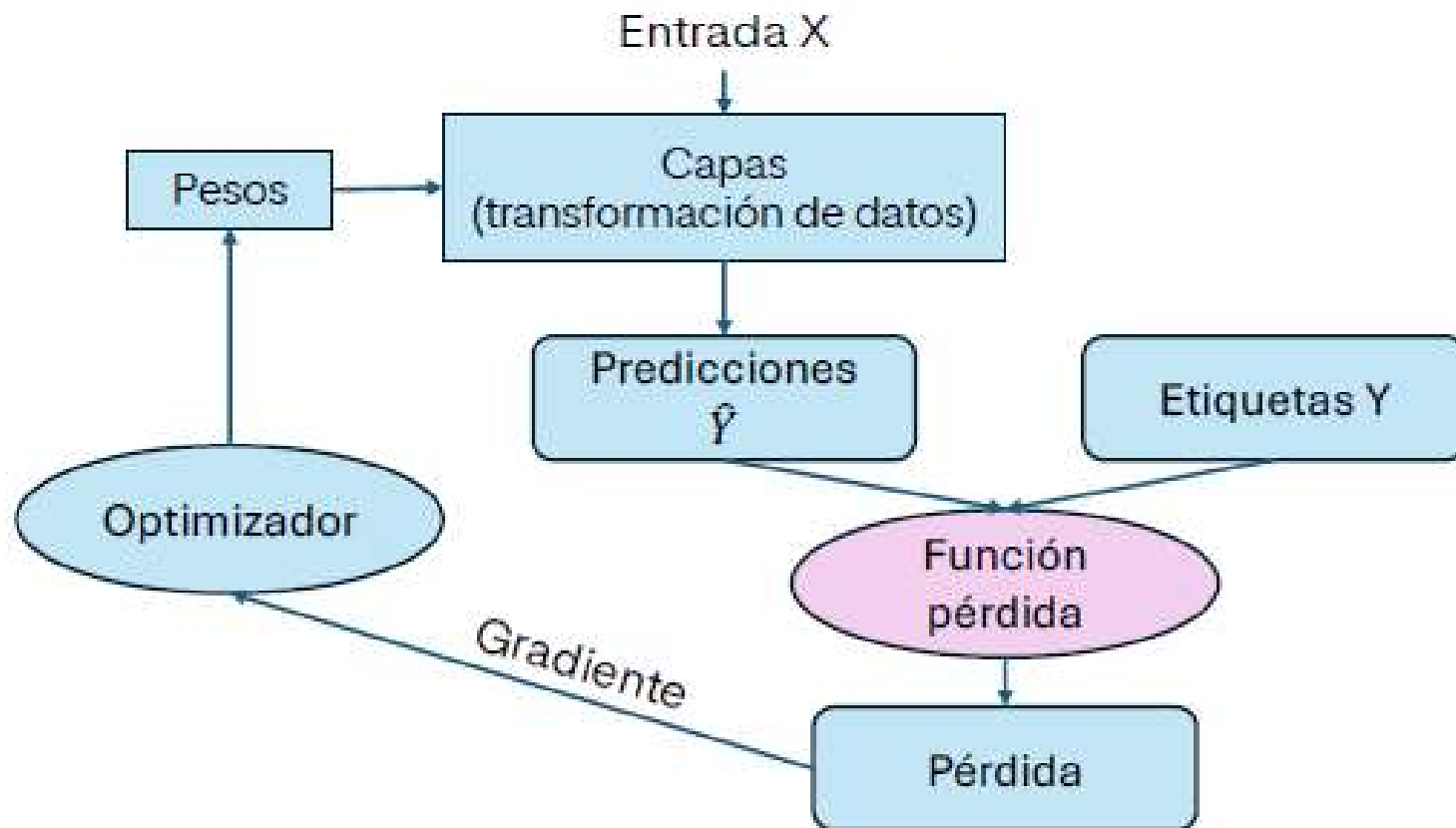
Redes Perceptrón Multicapa Entrenamiento

- El entrenamiento busca encontrar los pesos y sesgos de todas las capas de la red, de modo que la red asigne correctamente las entradas de ejemplo a sus targets (labels-etiquetas) asociados
- La función de pérdida cuantifica la discrepancia entre las predicciones del modelo y los valores reales (ground truth)
 - Función que guía el entrenamiento

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

Redes Perceptrón Multicapa Entrenamiento



Redes Perceptrón Multicapa

Funciones de Pérdida comunes

- Consideremos una red con I entradas y O salidas
- Un dataset de N datos para entrenar esta red tiene la forma

Entrada	Salida
$x_1^{(1)}, x_2^{(1)}, \dots, x_I^{(1)}$	$y_1^{(1)}, y_2^{(1)}, \dots, y_O^{(1)}$
$x_1^{(2)}, x_2^{(2)}, \dots, x_I^{(2)}$	$y_1^{(2)}, y_2^{(2)}, \dots, y_O^{(2)}$
$\vdots$	$\vdots$
$x_1^{(N)}, x_2^{(N)}, \dots, x_I^{(N)}$	$y_1^{(N)}, y_2^{(N)}, \dots, y_O^{(N)}$

Redes Perceptrón Multicapa Funciones de Pérdida comunes Problemas de Clasificación

- **Binaria: Binary cross-entropy**

$$\begin{aligned} J(\Theta) &= -\frac{1}{N} \sum_{n=1}^N \sum_{o=1}^O \left[y_o^{(n)} \cdot \log(\hat{y}_o^{(n)}) + (1 - y_o^{(n)}) \cdot \log(1 - \hat{y}_o^{(n)}) \right] = \\ &= -\frac{1}{N} \sum_{n=1}^N \sum_{o=1}^O \left[y_o^{(n)} \cdot \log(f_o(\mathbf{x}^{(n)}, \Theta)) + (1 - y_o^{(n)}) \cdot \log(1 - f_o(\mathbf{x}^{(n)}, \Theta)) \right] \end{aligned}$$

- **Multiclase: Categorical cross-entropy**

$$J(\Theta) = -\frac{1}{N} \sum_{n=1}^N y_o^{(n)} \cdot \log(\hat{y}_o^{(n)}) = -\frac{1}{N} \sum_{n=1}^N y_o^{(n)} \cdot \log(f_o(\mathbf{x}^{(n)}, \Theta))$$

Redes Perceptrón Multicapa Funciones de Pérdida comunes Problemas de Clasificación

Binary cross-entropy

- **Penaliza las predicciones seguras pero incorrectas:** gracias a los términos logarítmicos, la pérdida crece mucho si el modelo asigna probabilidad muy baja a la clase correcta
- **Continua y diferenciable**, lo cual facilita su optimización mediante métodos basados en gradiente
- **Sensible al desbalance entre clases:** se puede usar una variante ponderada
- **Interpretación probabilística:** equivale al negativo del logaritmo de la probabilidad asignada a la clase correcta
- **Se usa en clasificación binaria y conjunta con función sigmoide**

Redes Perceptrón Multicapa Funciones de Pérdida comunes Problemas de Clasificación

Categorical cross-entropy

- **Penaliza fuertemente** cuando el modelo asigna baja probabilidad a la clase correcta
- **Continua y diferenciable**, adecuada para optimización de gradiente
- **Enfoque probabilístico**: La salida representa una distribución de probabilidad sobre las clases, es decir

$$\sum_{o=1}^C \hat{p}_{o,n} = 1$$

- **Requisito de one-hot encoding**: asume que las etiquetas objetivo son vectores one-hot
- **Función de pérdida estándar** en clasificación multiclase con salida softmax

Redes Perceptrón Multicapa Funciones de Pérdida comunes Problemas de Regresión

- **Mean squared error (MSE) (Pérdida L2)**

$$J(\Theta) = \frac{1}{2} \sum_{n=1}^N \sum_{o=1}^O \left(y_o^{(n)} - \hat{y}_o^{(n)} \right)^2 = \frac{1}{2} \sum_{n=1}^N \sum_{o=1}^O \left(y_o^{(n)} - f_o(\mathbf{x}^{(n)}, \Theta) \right)^2$$

- **Mean absolute error (MAE) (Pérdida L1)**

$$J(\Theta) = \frac{1}{N} \sum_{n=1}^N \sum_{o=1}^O \left| y_o^{(n)} - \hat{y}_o^{(n)} \right| = \frac{1}{N} \sum_{n=1}^N \sum_{o=1}^O \left| y_o^{(n)} - f_o(\mathbf{x}^{(n)}, \Theta) \right|$$

Redes Perceptrón Multicapa Funciones de Pérdida comunes Problemas de Regresión

- **Huber: combina MSE y MAE**

$$J(\Theta) = \frac{1}{N} \sum_{n=1}^N \sum_{o=1}^O \mathcal{L}(y_o^{(n)}, \hat{y}_o^{(n)}) = \frac{1}{N} \sum_{n=1}^N \sum_{o=1}^O \mathcal{L}(y_o^{(n)}, f_o(\mathbf{x}^{(n)}, \Theta))$$

donde

$$\mathcal{L}(y_o^{(n)}, \hat{y}_o^{(n)}) = \begin{cases} \frac{1}{2} (y_o^{(n)} - \hat{y}_o^{(n)})^2 & \text{si } |y_o^{(n)} - \hat{y}_o^{(n)}| \leq \delta \\ \delta \left(|y_o^{(n)} - \hat{y}_o^{(n)}| - \frac{1}{2} \delta \right) & \text{en otro caso} \end{cases}$$

Redes Perceptrón Multicapa **Funciones de Pérdida comunes** **Problemas de Regresión**

Mean squared error (MSE)

- No negativa ($MSE \geq 0$): menores valores indican mejor ajuste del modelo
- Muy sensible a outliers: los errores grandes dominan la pérdida
- Continua, diferenciable y convexa (en modelos lineales): facilita la optimización por gradiente
- Dependiente de la escala: su magnitud depende de las unidades/escala de la variable objetivo
- Estándar en regresión
- Más fácil de minimizar que RMSE

Redes Perceptrón Multicapa **Funciones de Pérdida comunes** **Problemas de Regresión**

Mean absolute error (MAE)

- No negativa ($MAE \geq 0$): menores valores indican mejor ajuste del modelo.
- Menos sensible a outliers: penaliza linealmente
- Convexa (tiene mínimo global), pero no diferenciable en 0: en la práctica se usan subgradientes.
- Dependiente de la escala: para comparar entre problemas se usa Mean Absolute Percentage Error (MAPE) o Normalized MAE
- Altamente interpretable: se expresa en las mismas unidades que la variable objetivo

Redes Perceptrón Multicapa Funciones de Pérdida comunes Problemas de Regresión

Huber

- Menos sensible a outliers que MSE: penalización lineal para errores grandes
- Convexa y diferenciable en todas partes (pero no tiene segunda derivada en $|y - \hat{y}| = \delta$)
- Elección de δ : mediante validación cruzada.
 - δ pequeño $\rightarrow$ comportamiento como MSE
 - δ grande $\rightarrow$ comportamiento como MAE

Redes Perceptrón Multicapa **Funciones de Pérdida comunes** **Problemas de Regresión**

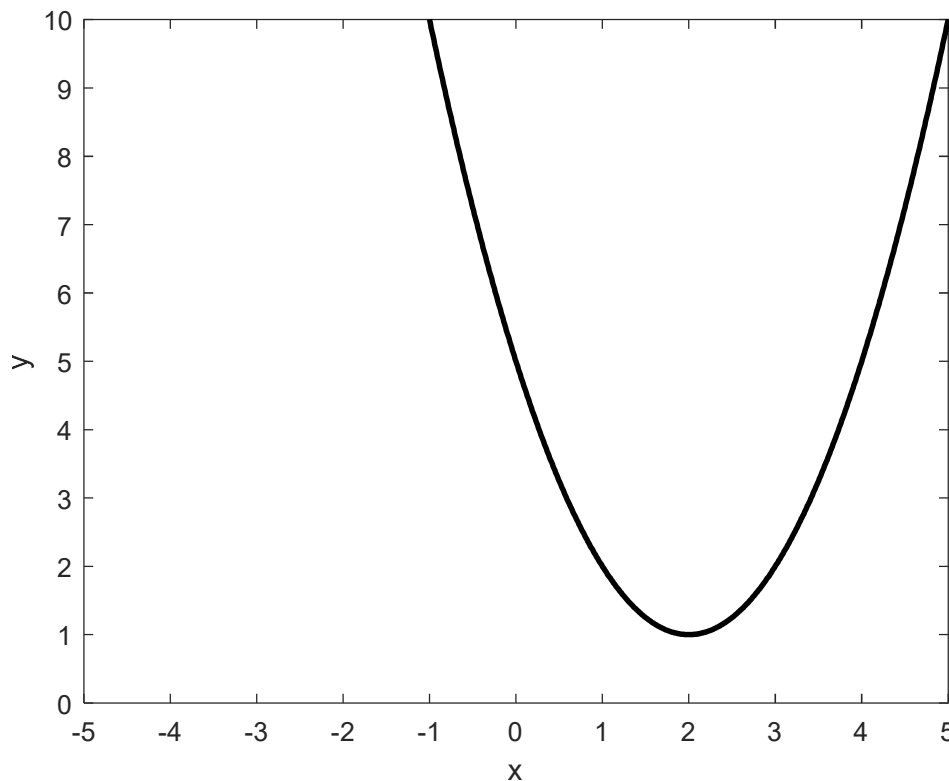
Comparación MSE-MAE-Huber

- MSE: alta sensibilidad a outliers, diferenciable, convergencia rápida cerca del mínimo
- MAE: robusta a outliers, no diferenciable en 0, gradiente constante
- Huber: balance configurable, diferenciable en todo el dominio

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

Mínimo de una función $y = f(x)$



$$y = f(x) = (x - 2)^2 + 1$$

Observe que el mínimo de $f(x)$ está en el punto (2,1)

Para hallar este mínimo

$$\frac{dy}{dx} = \frac{d}{dx} f(x) = 2(x - 2) = 0$$

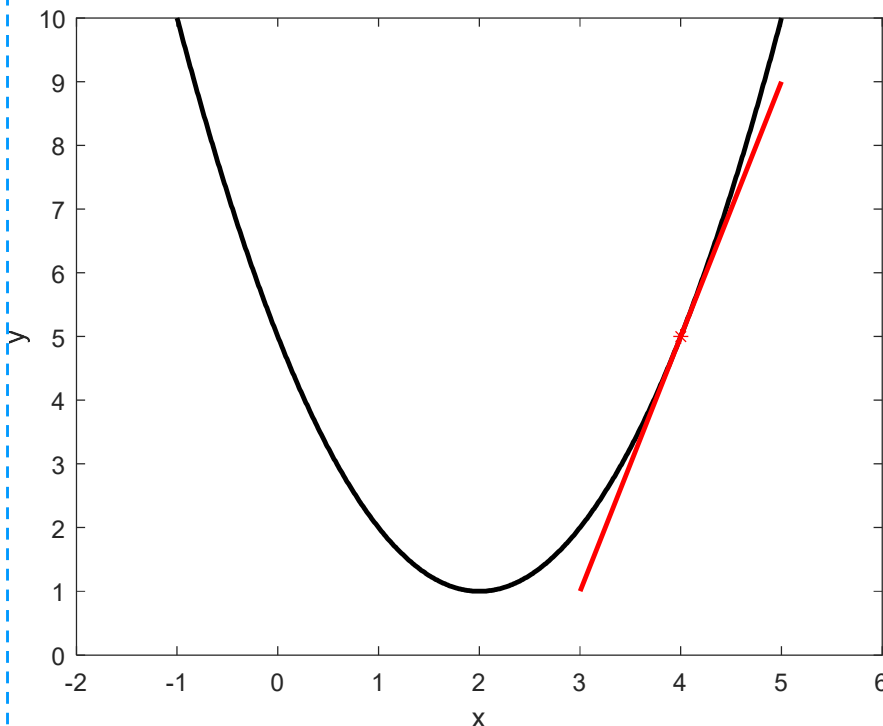
de donde $x = 2$ e

$$y = (2 - 2)^2 + 1 = 1$$

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

Mínimo de una función $y = f(x)$



También podemos hallar el mínimo por el método (iterativo) del descenso más rápido

$$x_{i+1} = x_i - \eta \left. \frac{d}{dx} f(x) \right|_{x_i}$$

$$x_{i+1} = x_i - \eta \cdot 2(x_i - 2)$$

partiendo desde un x_0 inicial

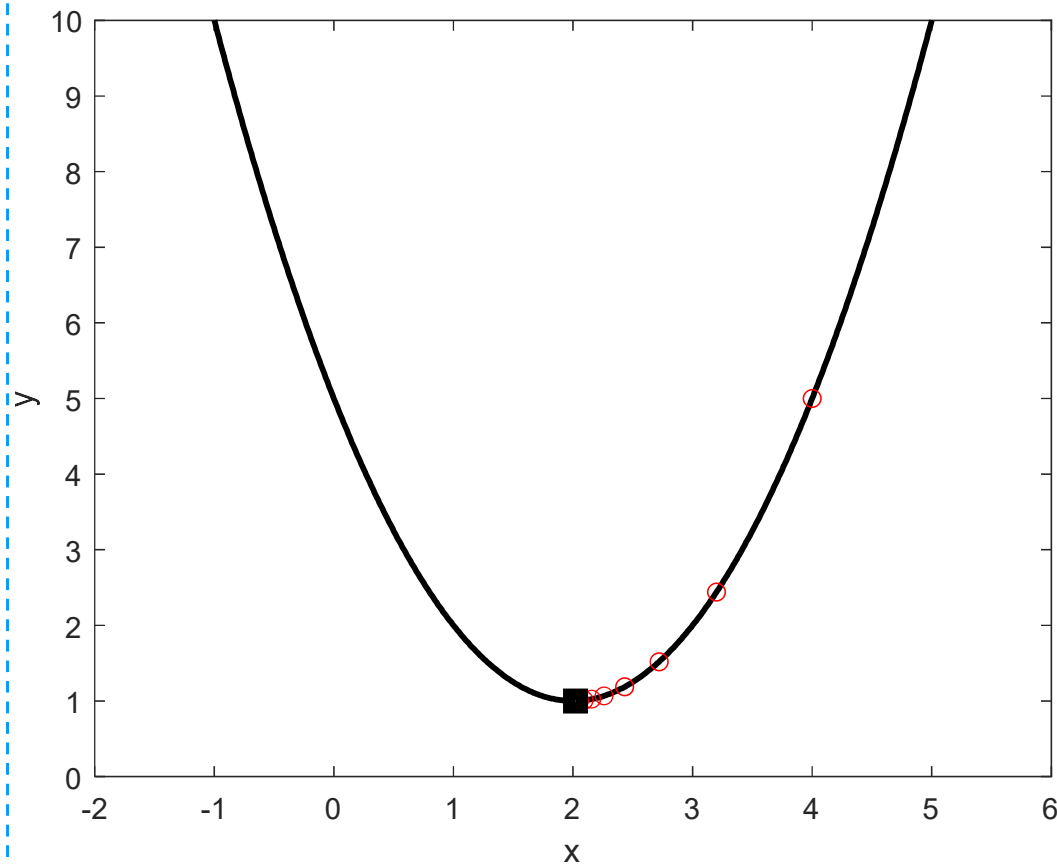
En $x_0 = 4$

$$\left. \frac{d}{dx} f(x) \right|_{x_0} = 2(4 - 2) = 4$$

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

Mínimo de una función $y = f(x)$



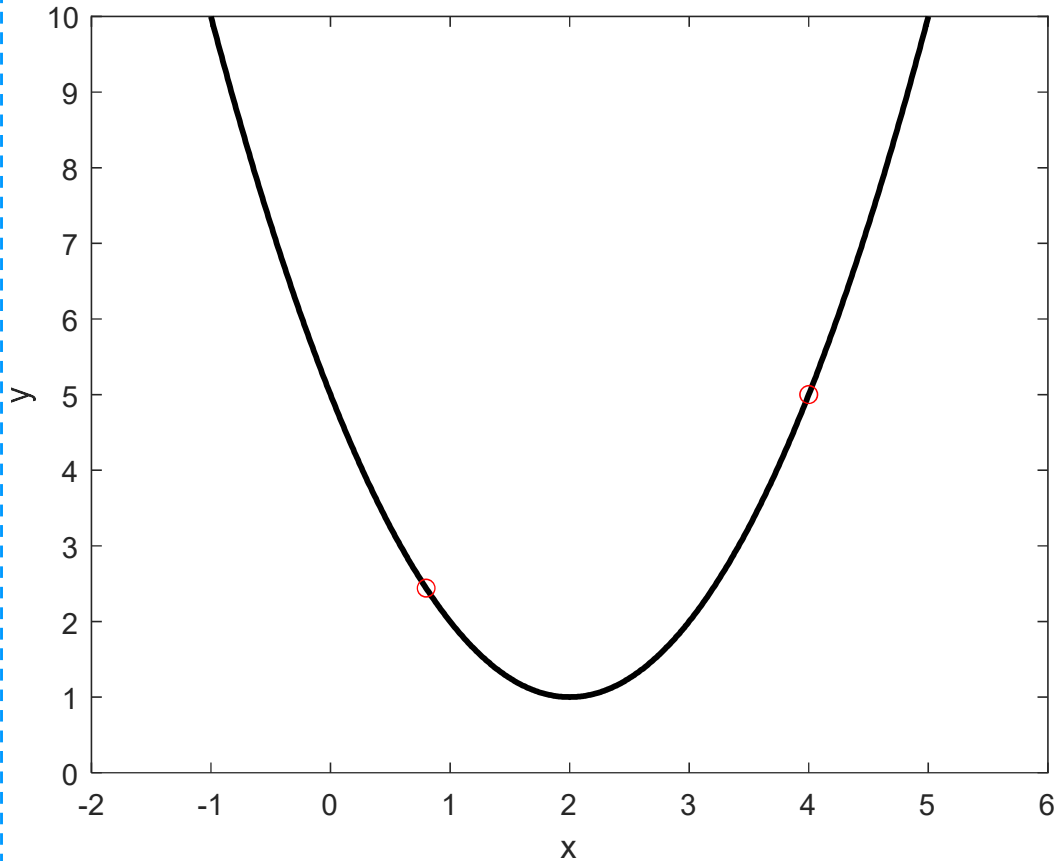
10 iteraciones con $\eta = 0.2$

i	x_i
0	4.0000
1	3.2000
2	2.7200
3	2.4320
4	2.2592
5	2.1555
6	2.0933
7	2.0560
8	2.0336
9	2.0202
10	2.0121

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

Mínimo de una función $y = f(x)$



1 iteración con $\eta = 0.8$

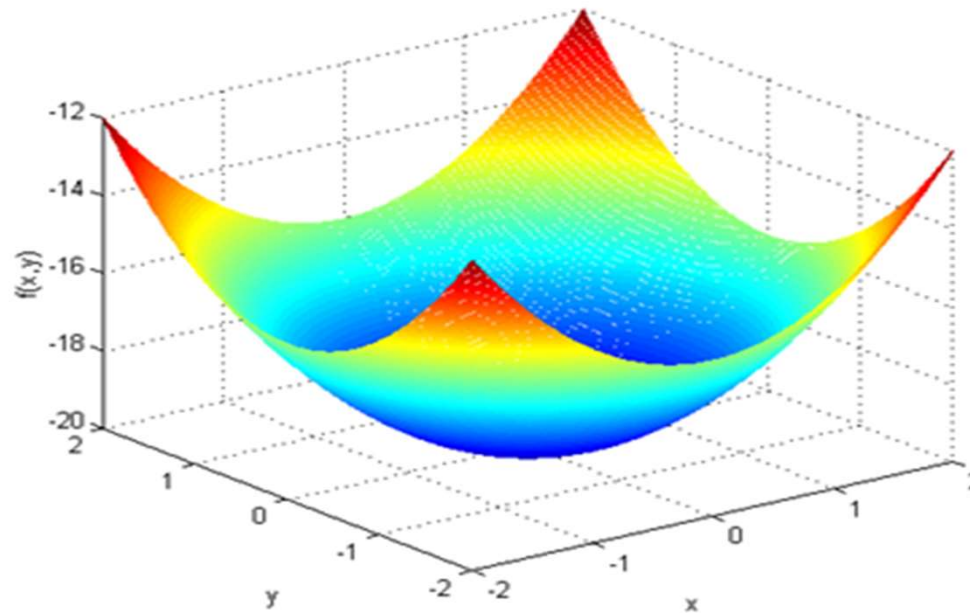
i	x_i
0	4.0
1	0.8

Método del descenso más rápido (Gradient descent)

EJEMPLO. Encuentre el mínimo de la función

$$f(x, y) = -20 + x^2 + y^2$$

Superficie de $f(x, y)$ en la región
 $\{(x, y) / x \in [-2, 2] \wedge y \in [-2, 2]\}$



Método del descenso más rápido (Gradient descent)

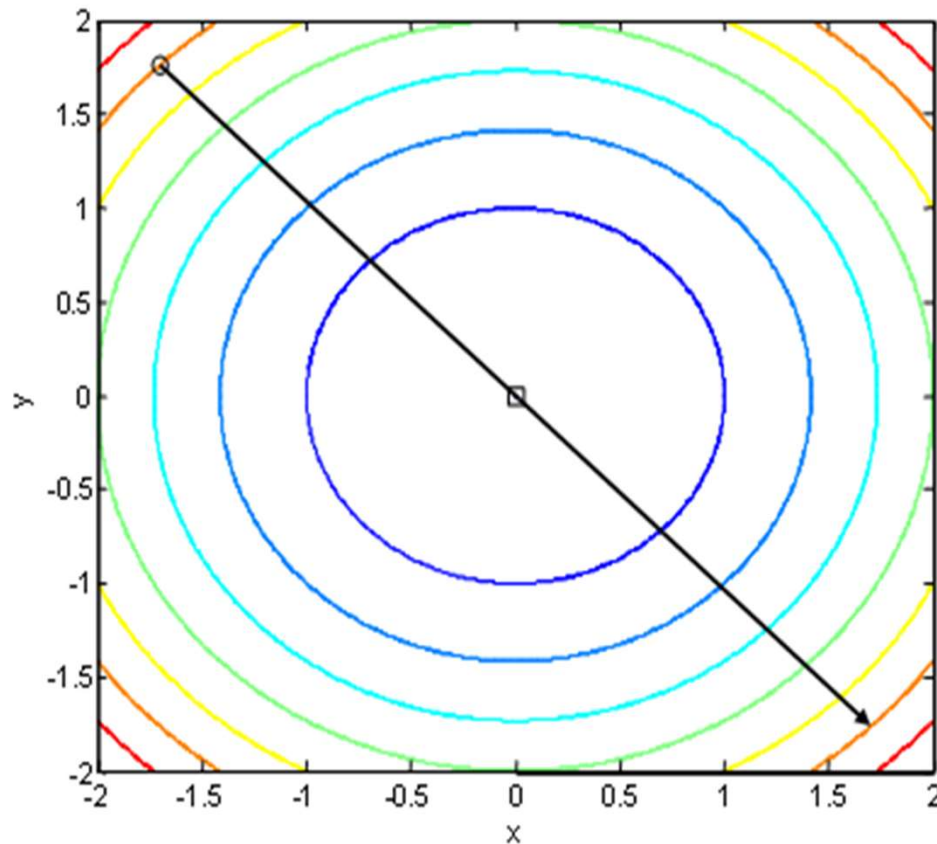
- Observe que la función tiene un único mínimo (unimodal) ubicado en $(0,0)$.
- El punto inicial se selecciona igual a $(-1.7, 1.7635)$, η es constante igual a 0.4 y $L = 20$.
- El gradiente de esta función en un punto cualquiera (x, y) es

$$\nabla f(x, y) = \left(\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y} \right) = (2x, 2y)$$

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

Método del descenso más rápido (Gradient descent)



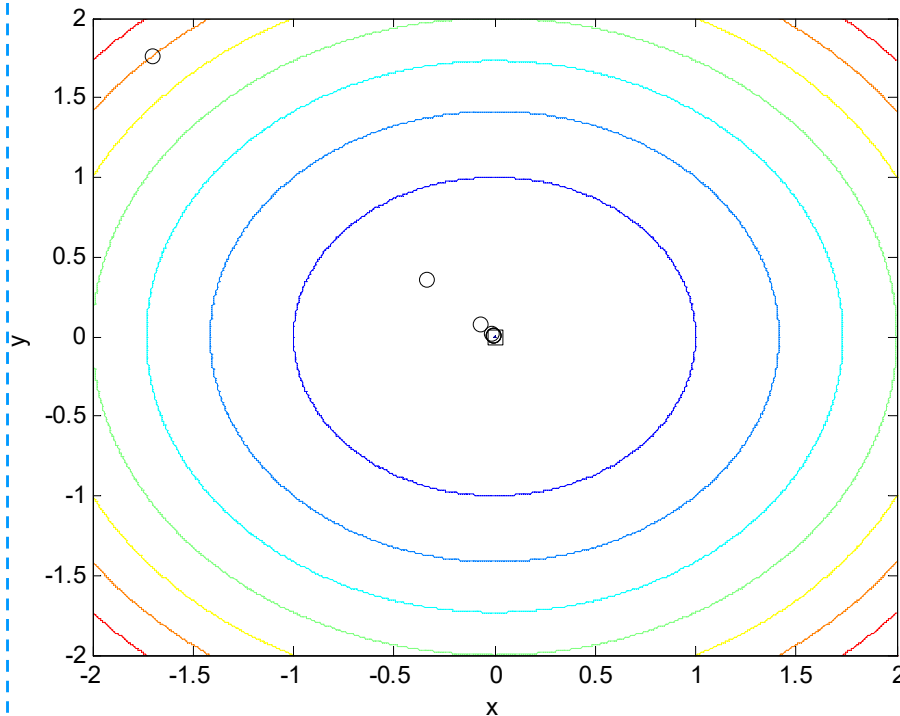
- Las curvas de nivel corresponden a círculos concéntricos con centro en el origen.
- El vector negativo del gradiente apunta hacia el mínimo.

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

Método del descenso más rápido (Gradient descent)

Evolución de gradient descent con $\eta = 0.4$



$$\begin{aligned} x(k+1) &= x(k) - \eta \left. \frac{\partial f(x, y)}{\partial x} \right|_{\substack{x(k) \\ y(k)}} \\ &= x(k) - \eta[2x(k)] \end{aligned}$$

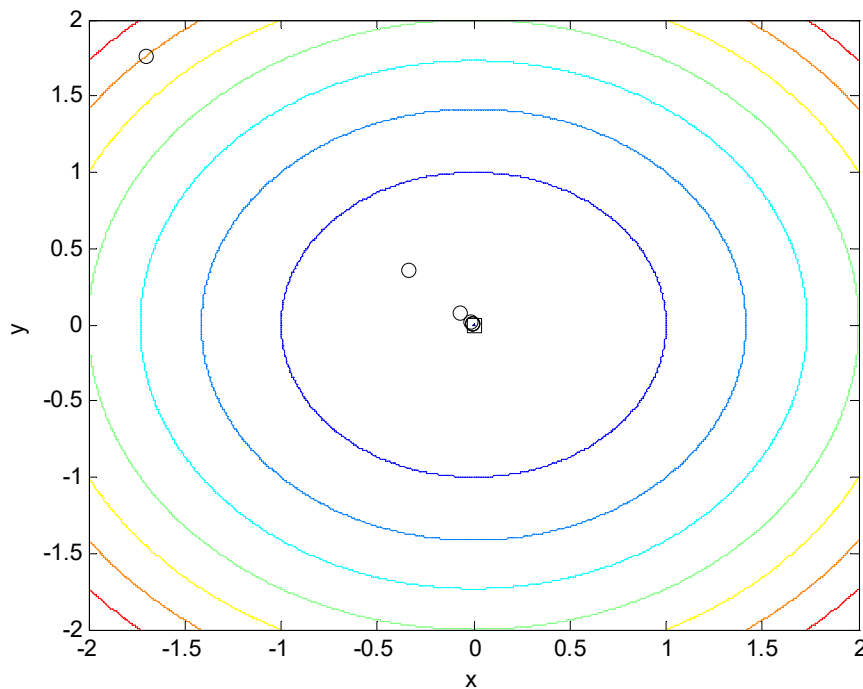
$$\begin{aligned} y(k+1) &= y(k) - \eta \left. \frac{\partial f(x, y)}{\partial y} \right|_{\substack{x(k) \\ y(k)}} \\ &= y(k) - \eta[2y(k)] \end{aligned}$$

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

Método del descenso más rápido (Gradient descent)

Evolución de gradient descent con $\eta = 0.4$



k	$x(k)$	$y(k)$
0	-1.7	1.7635
1	-0.34	0.3527
2	-0.068	0.0705
3	-0.0136	0.0141
4	-0.0027	0.0028
5	-0.0005	0.0006
6	-0.0001	0.0001

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

Entrenamiento de una RPMC usando Backpropagation

- Desarrollado por Rumelhart, Hinton y Williams: publicaron el paper seminal "*Learning representations by back-propagating errors*" en Nature (1986), que popularizó el algoritmo y demostró su utilidad para entrenar redes multicapa
- Anteriormente, Paul Werbos (1974) describió backprop en su tesis doctoral en Harvard, aplicándola a redes neuronales. Fue el primero en hacerlo claramente, pero el trabajo pasó desapercibido durante años

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

Entrenamiento de una RPMC usando Backpropagation

- Consideremos una red con I entradas, una capa oculta de H unidades y O salidas
- Un dataset de N datos para entrenar esta red tiene la forma

Entrada	Salida
$x_1^{(1)}, x_2^{(1)}, \dots, x_I^{(1)}$	$y_1^{(1)}, y_2^{(1)}, \dots, y_O^{(1)}$
$x_1^{(2)}, x_2^{(2)}, \dots, x_I^{(2)}$	$y_1^{(2)}, y_2^{(2)}, \dots, y_O^{(2)}$
$\vdots$	$\vdots$
$x_1^{(N)}, x_2^{(N)}, \dots, x_I^{(N)}$	$y_1^{(N)}, y_2^{(N)}, \dots, y_O^{(N)}$

Entrenamiento de una RPMC usando Backpropagation

- El algoritmo utiliza el método del descenso más rápido para minimizar la función de pérdidas
- Por ejemplo, en el caso de un problema de regresión

$$J(\Theta) = \frac{1}{2} \sum_{n=1}^N \sum_{o=1}^O \left(y_o^{(n)} - f_o(\mathbf{x}^{(n)}, \Theta) \right)^2$$

donde Θ es un vector que contiene todos los pesos de la red

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

Entrenamiento de una RPMC usando Backpropagation

- El peso que conecta la unidad h de la capa oculta y la unidad o de la capa de salida se actualiza como

$$w_{oh}(k+1) = w_{oh}(k) - \eta \frac{\partial J}{\partial w_{oh}}; \quad o = 1, \dots, O; \quad h = 1, \dots, H$$

donde η es el learning rate

- Puede demostrarse que para SGD

$$\frac{\partial J}{\partial w_{oh}} = - \left(y_o^{(n)} - \hat{y}_o^{(n)} \right) \left[f'_o(\sigma) |_{\sigma=a_o} \right] h_h$$

donde $f'_o(\sigma)$ es la derivada de la función de activación de las unidades de salida y $a_o = \sum_{h=1}^H w_{oh} h_h$

Entrenamiento de una RPMC usando Backpropagation

- El peso que conecta la unidad i de entrada y la unidad h de la capa oculta se actualiza como

$$v_{hi}(k + 1) = v_{hi}(k) - \eta \frac{\partial J}{\partial v_{hi}}; h = 1, \dots, H; i = 1, \dots, I$$

donde η es el learning rate

- Puede demostrarse que para SGD

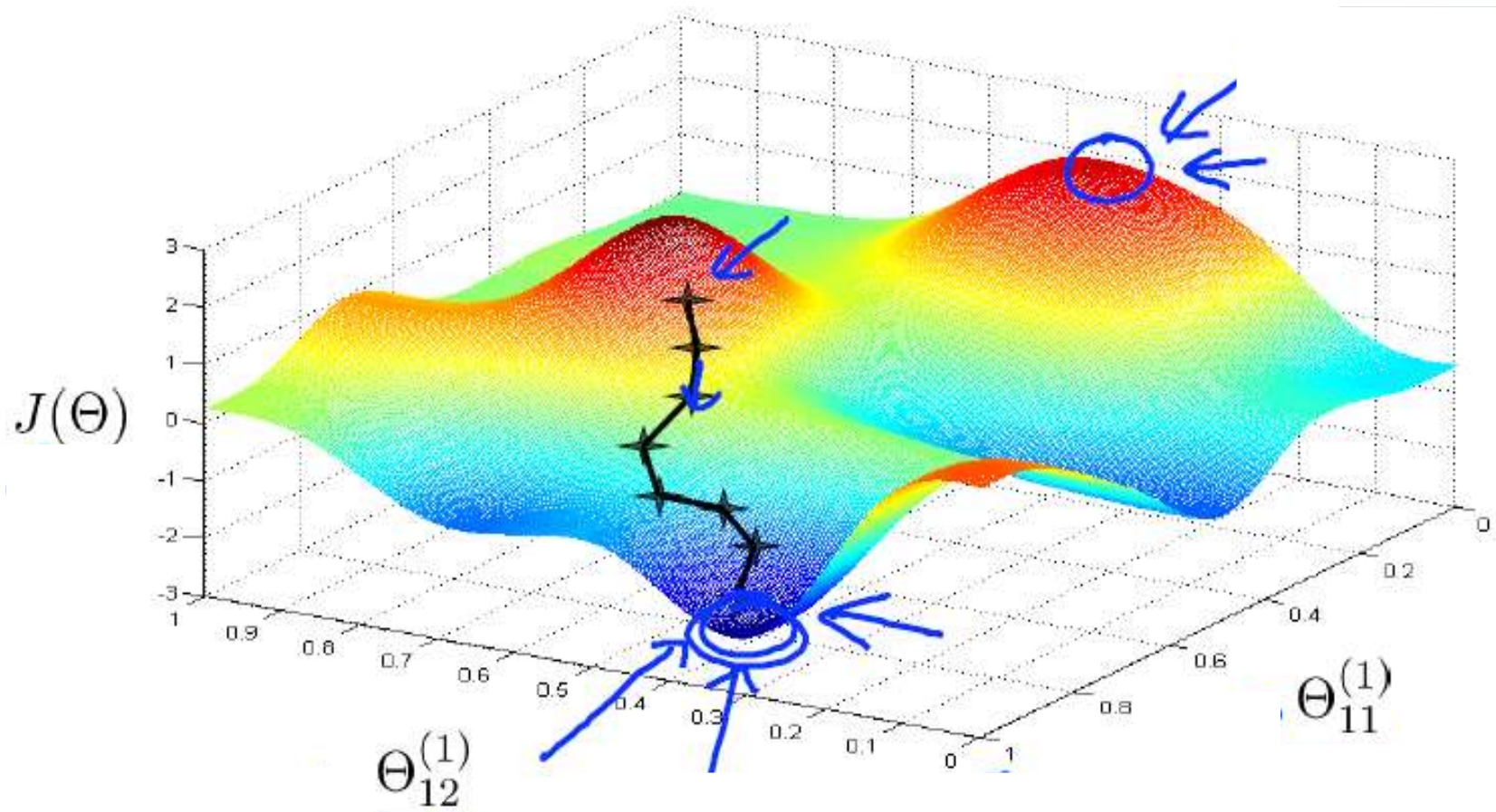
$$\frac{\partial J}{\partial v_{hi}} = - \sum_{o=1}^O \left(y_o^{(n)} - \hat{y}_o^{(n)} \right) \left[f'_o(\sigma) \Big|_{\sigma=a_o} \right] w_{oh} \left[f'_h(\sigma) \Big|_{\sigma=b_h} \right] x_i^{(n)}$$

donde $f'_h(\sigma)$ es la derivada de la función de activación de las unidades de la capa oculta y $b_h = \sum_{i=0}^I v_{hi} x_i^{(n)}$

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

Entrenamiento de una RPMC usando Backpropagation



INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

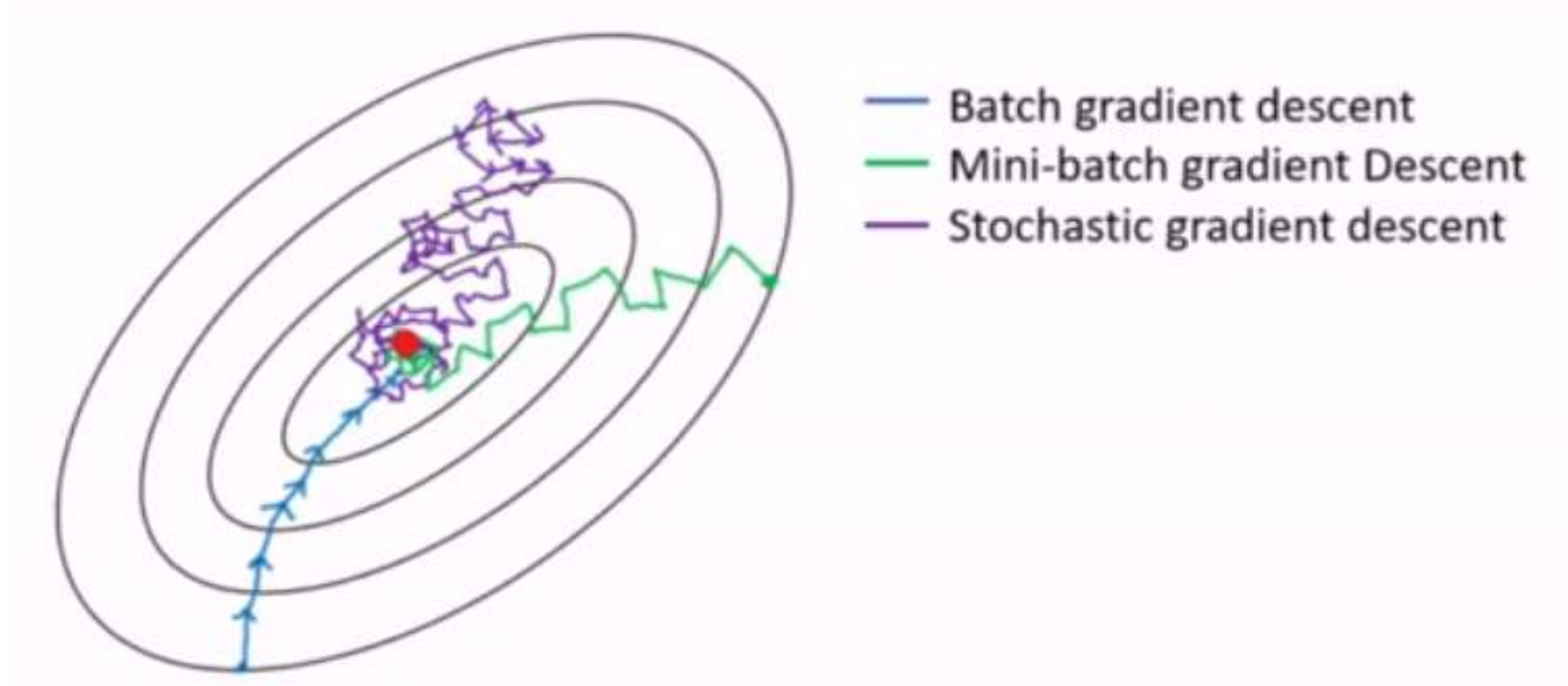
Entrenamiento de una RPMC usando Backpropagation

- Las ecuaciones de actualización de los pesos pueden implementarse de forma matricial, que es más eficiente desde el punto de vista computacional
- Los pesos pueden actualizarse luego que se presenta
 - Todos los vectores de entrenamiento (versión batch)
 - Un subconjunto de vectores de entrenamiento (versión mini-batch)
 - Un vector de entrenamiento (SGD, stochastic gradient descent)

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

Entrenamiento de una RPMC usando Backpropagation



INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

Entrenamiento de una RPMC usando Backpropagation

- Si la red se está entrenando para resolver un problema de clasificación, se sigue un procedimiento similar para minimizar la función de pérdida
 - binary cross-entropy (problemas de clasificación binaria)
 - categorical cross-entropy (problemas multiclase)

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

Capacidad de Representación

¿Cuántas regiones lineales crea una red shallow?

- Con H neuronas ReLU en $1D$: hasta $H + 1$ regiones lineales
- Con H neuronas ReLU en D dimensiones: $O(H^D)$ regiones lineales

Número de parámetros vs. capacidad

- Con P parámetros totales, una red shallow tiene capacidad $\sim O(P)$ regiones
- Una red profunda con los mismos parámetros puede tener capacidad exponencialmente mayor

Red Shallow vs Red Deep

- Red shallow: H neuronas $\rightarrow O(H)$ regiones en $1D$
- Red de L capas:
 - En $1D$: H neuronas por capa $\rightarrow O(H^L)$ regiones
 - En D dimensiones: H neuronas por capa $\rightarrow O(H^{DL})$ regiones

La profundidad aumenta capacidad de forma exponencial con el número de capas

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

Capacidad de Representación

Conclusión clave

- El teorema de aproximación universal garantiza existencia, no eficiencia
- Representar algunas funciones con una red shallow requiere significativamente más parámetros

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

Limitaciones de las Redes Shallow

Problema 1: Ineficiencia de parámetros

- Para representar funciones 'compuestas' ($f = g \circ h$), una red shallow necesita exponencialmente más neuronas que una red profunda
- Ejemplo: reconocer una cara $\rightarrow$ bordes $\rightarrow$ partes $\rightarrow$ rostro. Una sola capa no puede aprovechar esta jerarquía
- Funciones que se representan eficientemente con depth L requieren exponencialmente más unidades con depth $L - 1$

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

Limitaciones de las Redes Shallow

Problema 2: No captura estructura jerárquica

- El mundo real tiene estructura de composición:
 - Objetos = partes; oraciones = palabras; señales = frecuencias
- Las redes profundas aprenden representaciones jerárquicas de forma natural

Problema 3: Generalización

- Redes muy anchas (shallow con H grande) tienden a sobreajustar con datos limitados
- La inducción implícita de la profundidad actúa como regularizador

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

Motivación para la Profundidad

Argumento 1: Eficiencia exponencial

- Una red de L capas con H neuronas representa $O(H^L)$ regiones lineales
- Una red shallow necesita H^L neuronas para el mismo poder expresivo

Argumento 2: Composición de funciones simples

- Cada capa aprende una transformación del espacio de representación
- Capas sucesivas construyen abstracciones progresivamente más complejas
- Análogo al razonamiento humano: de lo concreto a lo abstracto

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

Motivación para la Profundidad

Argumento 3: Evidencia empírica en visión

- Capa 1 → bordes y gradientes de color
- Capa 2 → texturas y esquinas
- Capa 3 → partes de objetos
- Capa 4+ → objetos completos y conceptos

En resumen:

- 'Adding more layers to a network generally leads to better performance'
- No linealidad iterada crea fronteras de decisión arbitrariamente complejas
- La profundidad transforma representaciones no linealmente separables en linealmente separables

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad II

De Shallow a Deep

